Practice 15. Autocomplete System

[CSE2010] Data Structures

Department of Data Science

Practice 13 Solution (Python)

- Main
  - Creates a temporary array and calls mergeSort()
  - Calls quicksort()

```python
if __name__ == "__main__":
  if len(sys.argv) != 3:
    raise Exception("Correct usage: [program] [input] [output]")
  with open(sys.argv[1], 'r') as inFile:
    lines = inFile.readlines()
  with open(sys.argv[2], 'w') as outFile:
    i = 0
    while i < len(lines):
      words = lines[i].split()
      op = words[0]
      if len(words) != 2:
        raise Exception("Error: invalid input")
      size = int(words[1])
      i += 1
      arr = readInput(lines[i], size)
      if op == MERGE_SORT:
        tempArray = [0] * size
        mergeSort(arr, tempArray, 0, size - 1)
      elif op == QUICK_SORT:
        quickSort(arr, 0, size - 1)
      else:
        raise Exception("Undefined operator")
      write(outFile, arr)
      i += 1
```

Practice 13 Solution (Python)

- Merge sort
  - Needs auxiliary space for a temporary array

```python
def merge(arr, tempArr, lPos, rPos, rEnd):
  lEnd = rPos - 1
  tPos = lPos
  size = rEnd - lPos + 1
  while lPos <= lEnd and rPos <= rEnd:
    if arr[lPos] >= arr[rPos]:
      tempArr[tPos] = arr[lPos]
      tPos += 1
      lPos += 1
    else:
      tempArr[tPos] = arr[rPos]
      tPos += 1
      rPos += 1
  while lPos <= lEnd:
    tempArr[tPos] = arr[lPos]
    tPos += 1
    lPos += 1
  while rPos <= rEnd:
    tempArr[tPos] = arr[rPos]
    tPos += 1
    rPos += 1
  for i in range(size):
    arr[rEnd] = tempArr[rEnd]
    rEnd -= 1

def mergeSort(arr, tempArr, l, r):
  if l >= r: return
  mid = (l + r) // 2
  mergeSort(arr, tempArr, l, mid)
  mergeSort(arr, tempArr, mid + 1, r)
  merge(arr, tempArr, l, mid + 1, r)
```

- Quicksort
  - In-place: constant space

```python
def partition(arr, p, r):
  x = arr[r]
  i = p - 1
  for j in range(p, r):
    if arr[j] >= x:
      i += 1
      arr[i], arr[j] = arr[j], arr[i]
  arr[r], arr[i + 1] = arr[i + 1], arr[r]
  return i + 1

def quickSort(arr, p, r):
  if p >= r: return
  q = partition(arr, p, r)
  quickSort(arr, p, q - 1)
  quickSort(arr, q + 1, r)
```

TODO

- Write a program that solves a below problem.
- **Problem**
  - Implement an efficient autocomplete system. Given a query string s and a set of all possible query strings, return all strings in the set that have s as a prefix.
- Example
  - Input: words=**`[cat, dog, deer, deal]`**, s=**`de`**
  - Output: **`[deer, deal]`**

Every Two Lines Represent an Operation in Input

- **T<space>[number_of_words]**

  [word]
  ⋮ **[number_of_words]**
  [word]

  - Parse input words.
- **A<space>[prefix]**
  - Write to the output file all the words that have **[prefix]** as a prefix.

Input and Output Examples

- Input File

```
T 4
cat
dog
deer
deal
A de
A dea
```

Output File

```
deer deal
deal
```

Running Example

```
[hjkim@localhost tries]$ cat input.txt
T 4
cat
dog
deer
deal
A de
A dea
[hjkim@localhost tries]$ g++ -std=c++11 autocomplete.cpp -o autocomplete
[hjkim@localhost tries]$ ./autocomplete input.txt output.txt
[hjkim@localhost tries]$ cat output.txt
deal deer
deal
[hjkim@localhost tries]$
```

Submission Guideline

- Submission: **source code, an input file, an output file**
  - Where: Assignments in LMS
  - Deadline: **23:59, June. 18th (Tuesday)**

Naïve Solution

```
def naiveAutocomplete(words, s):
  out = []
  for word in words:
    if word.startswith(s):
      out.append(word)
  return out
```

- $O(n)$ time where n is the number of words
- Can we do better than this on average?
- Hints

1. Implement a trie.
2. Make use of a trie to get every word starting with a given prefix.